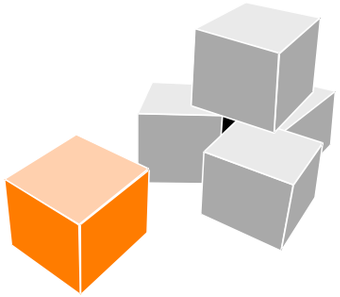
Concepts of Programming Languages

This Course in a Nutshell

Prof. Dr. Mira Mezini
Dr. Ragnar Mogk



Software Technology Group
TU Darmstadt | FB Informatik

Topics of Today‘s Lecture

- Why study programming languages (PLs)?
- What PL aspects will we study and how?
- A glimpse of COPL’s working language
- COPL’s teaching approach
- Administrativa

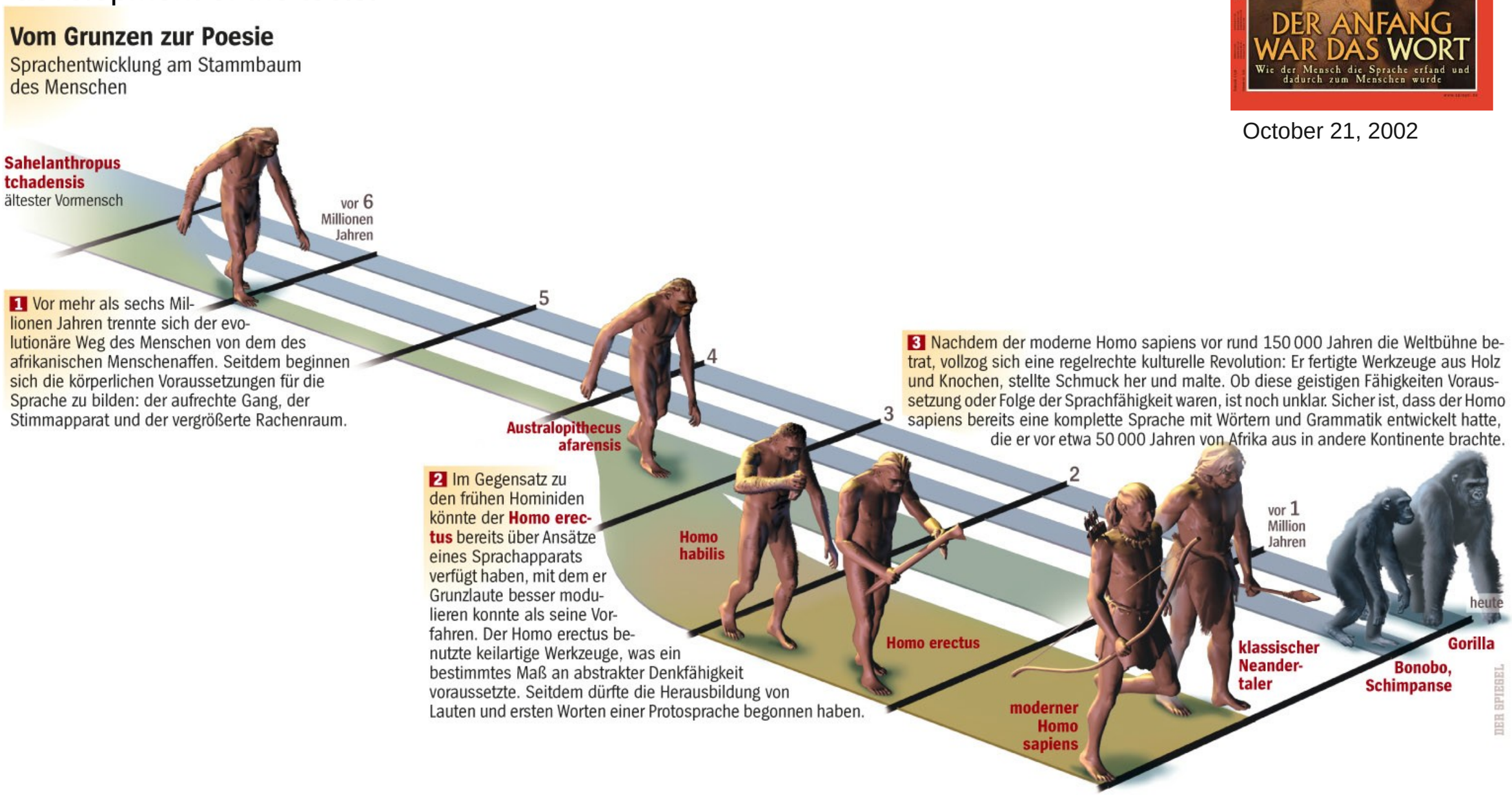
WHY STUDY PLs

On the Power of Language

... before the emergence of grammatically structured languages
… the tools of Homos Erectus have hardly developed for 2 million years...

Only after the emergence of the language, one observes also a rapid development of the tools.

October 21, 2002

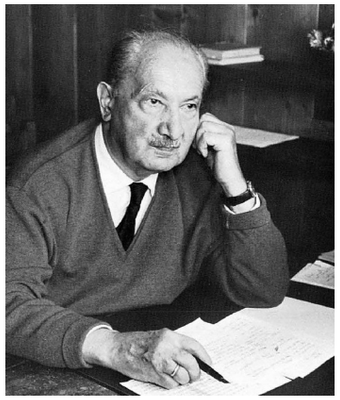


On the Power of Language

Der Mensch ist nur Mensch durch Sprache; um aber Sprache zu erfinden, müsste er schon Mensch sein.

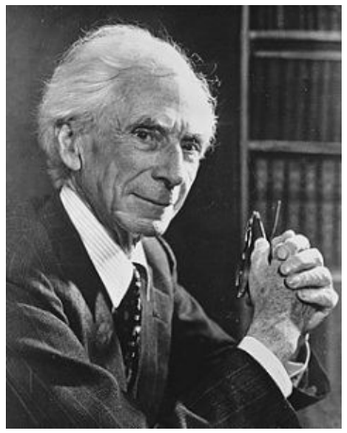
Die Sprache ist das bildende Organ des Gedanken.

Wilhelm von Humboldt (1767 - 1835), German philosopher, linguist, …



Die Sprache ist das Haus des Seins

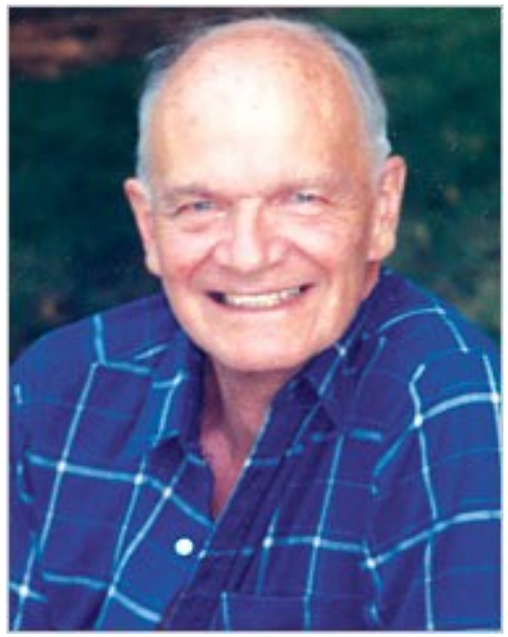
Martin Heidegger (1889–1976), German philosopher



Language serves not only to express thoughts, but to make possible thoughts which could not exist without it.

Bertrand Russell (1872 - 1970), British philosopher

On the Power of Language



When we talk about intelligence, we are basically talking about spin-offs ... of language.

Derek Bickerton (1926 – 2018), Linguist

Sprache ist das entscheidende Instrument des Bewusstseins

Angela Friederici (1952), Neuroscientist

Speculative Claim

The development of computer languages is decisive also for the development of "computational intelligence"!

Why Study PLs?

- **Powerful instruments to control complexity in design**
  - To programmers, they are the ultimate human-computer interface
  - Not only a means for instructing a computer to perform tasks
  - Primarily frameworks within which we precisely capture and organize ideas about problem domains!

- "Better" languages
  - increase our ability to deal with complex problems by providing ways to describe things more directly.
  - enable more productive software development

```
       i = 1
TEST:  if i < 4
       then goto BODY
       else goto END

BODY:  print(i)
       i = i + 1
       goto TEST
END:
```

```
i = 1
while ( i < 4 ) {
    print(i)
    i = i + 1
}
```

Why Study PLs?

Modelling techniques for computer systems **and technologies for developing new programming languages** overlap each other.

- Insights into PL design are applicable to **API** design Many computational domains can be considered PLs

- Example: Solving polynomial equations
  - Specify the computation in some notation (API)
  - Implementation of the API embodies computation rules for polynomials
  - Report result in some notation

WHAT LANGUAGE ASPECTS DO WE STUDY, AND HOW?

Four Aspects of Every PL

What are they?

1. Syntax: structure of programs
2. Semantics: meaning associated to syntax
3. Libraries: reusable computations
4. Idioms used by programmers of that language

Which of them is the most important for the study of PLs?

Which Fragments Are Most Similar?

`a[25] + 5`	Java
`(+ (vector-ref a 25) 5)`	Scheme
`a[25] + 5`	C

Hint: Think about adding elements out of the array boundaries…

Syntax Doesn’t Tell Anything About Behavior

- Syntax is important! At least in as much as it allows programmers to say more by writing less.
- But it get into our way in studying languages.
- We will use a **uniform syntax for all <u>languages that we will implement</u>**.

How Do We Study Semantics?

We implement the semantics of PL concepts we study!

- We build **evaluators**: programs that map programs to answers.
- By telling the computer what it means to map language concept to answers, we thoroughly understand the meaning of the concepts ourselves.

There are broadly speaking two kinds of language evaluators. What are those?

Two Kinds of Language Evaluators

interpreter :: $\text{Program}_L \rightarrow \text{Value}$

- An interpreter consumes a program and **simulates its execution**.
- The interpreter does what we would expect “running the program” should do.

compiler :: $\text{Program}_L \rightarrow \text{Program}_T$

- A compiler consumes a program and produces another program.
- The output program must then be further evaluated.

Which one do we use for our purpose and why?

Interpreters are Our Choice

We write interpreters to define, study, explain PL concepts **Interpreter Semantics**

- An interpreter is often easier to write than a compiler can reuse from the language in which we write the interpreter.
- Debugging an interpreter can be easier than debugging a compiler.
- Interpreters are a “baseline” implementation technology that is broadly accessible. Compilers can often take an entire course of study.

This is a course on core concepts of PLs not one on methods and technologies for implementing PLs!

Discussion on Terminology

It is common that people speak of “interpreted languages” and “compiled languages”.

- **Q**: Do the terms „Interpreted Languages“ and „Compiled Languages” make sense?
- **Q**: Is the distinction between interpreter and compiler evaluators strict?
- **Q:** Is the interface that a language implementation presents to users an indicator of what kind of implementation one is seeing?

Definitional Interpreters

- **An interpreter that defines a language cannot be "wrong".** It **defines** the meaning.
- Assigning '+' another meaning than addition is not wrong, at most it is unconventional. **Example?**
- One can speak only about the correctness of the interpreter relative to a formal specification, if one exists.

Some Interpreter Terminology

- **Host language** (or **base language**)

  The language, in which we implement the interpreter.

  In our case: Scala

- **Interpreted language** (or **object language**)

  The language that the interpreter evaluates.

  We assume that we already understand the host language.

- **Meta-interpreter** or **Meta-circular interpreter**

  host language = interpreted language

Other Ways to Define Semantics?

- Informal specs and language surveys
- Formal specifications:
  - Operational, denotational, axiomatic semantics
  - If at all, we'll only take a brief look at formal semantics
- Operational semantics are a close cousin of interpreter semantics – op semantics uses mathematical notation rather than code to define semantics
- We'll interleave interpreter semantics and language surveys

OUR HOST LANGUAGE

A Subset of SCALA

- Design started in 2001 at École Polytechnique Fédérale de Lausanne (EPFL)
- Public release in early 2004 on the Java platform
- A second version (2.0) followed in March 2006
- Current release in September 2023

The Scala language is maintained by

The Scala Center is supported by

Driving Forces for Scala's Design

Two hypotheses:

1. PLs for component software should be **scalable**
   - The same concepts describe small and large parts
   - Rather than adding lots of primitives, focus on abstraction, composition, and decomposition

2. Unification of OO and functional programming can provide scalable support for components

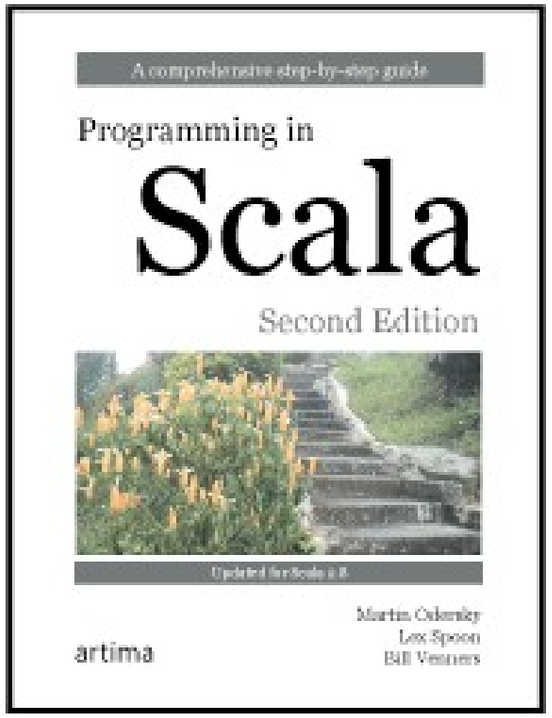
Adoption is key for testing the hypotheses => Scala interoperates with Java

Scala's Adoption

The Scala Language

- We will present only as much Scala as needed.
- For learning Scala, refer to books and online courses:

  https:// www.coursera.org/course/progfun



More books:

http://www.scala-lang.org/node/959

Quick entrance for Java Programmers:

http://docs.scala-lang.org/tutorials/scala-for-java-programmers.html

A Tiny Glimpse of Scala

- Everything is an object: Numbers and functions are objects
- Classes, inheritance, case classes and pattern matching

Scala Functions are Objects

```scala
object Timer {
  def oncePerSecond (callback: () => Unit) =
       while (true) {
         callback();
         Thread sleep 1000 }

  def timeFlies() =
      println("time flies like an arrow...")

  def main(args: Array[String]) =
      oncePerSecond(timeFlies)
}
```

What does the code do?

Someone wants to try explaining (guessing)?

Scala Classes

class parameters (primary constructor)

```
class ComplexNumbers(real: Double, imaginary: Double) {
  def re = real
  def im = imaginary

  override def toString() =
    "" + re + (if (im < 0) "" else "+") + im + "i"
}

object ComplexNumbers {
  def main(args: Array[String]) =
    val c = new ComplexNumbers(1.2, 3.4)
    println(c.toString)
    println("imaginary part: " + (c.im))

}
```

type inference in the definition of re and im

parameterless methods look like field declarations

postfix operator notation for parameterless method calls

Algebraic Data Types (ADTs) in Scala

ADTs are like building blocks for creating complex data structures.

Product Types

- "AND" combinations: Like a recipe that needs all ingredients
- Example: A person with a name AND an address
- Similar to multiplication: possibilities multiply

Sum Types

- "OR" combinations: Like choosing from a menu
- Example: A shape that can be a circle OR square OR triangle
- Similar to addition: possibilities add up

Q: How would you model binary trees as ADTs?

Algebraic Data Types (ADTs) in Scala

ADTs are useful useful because they let you:

- Build precise models of your data
- Catch errors early (at compile-time)
- Write cleaner code with pattern matching Esp. valuable for processing recursive (e.g., tree) structures

Just as you can build complex structures with Lego pieces, ADTs let you create complex data types from simple ones, making code more expressive and safer

Case Classes for Algebraic Data Types (ADTs)

Scala's case classes model regular ADT as objects and enable pattern matching on such objects

```
abstract class Tree

case class Leaf(n: Int) extends Tree
case class Node(left: Tree, right: Tree) extends Tree
```

Pattern Matching

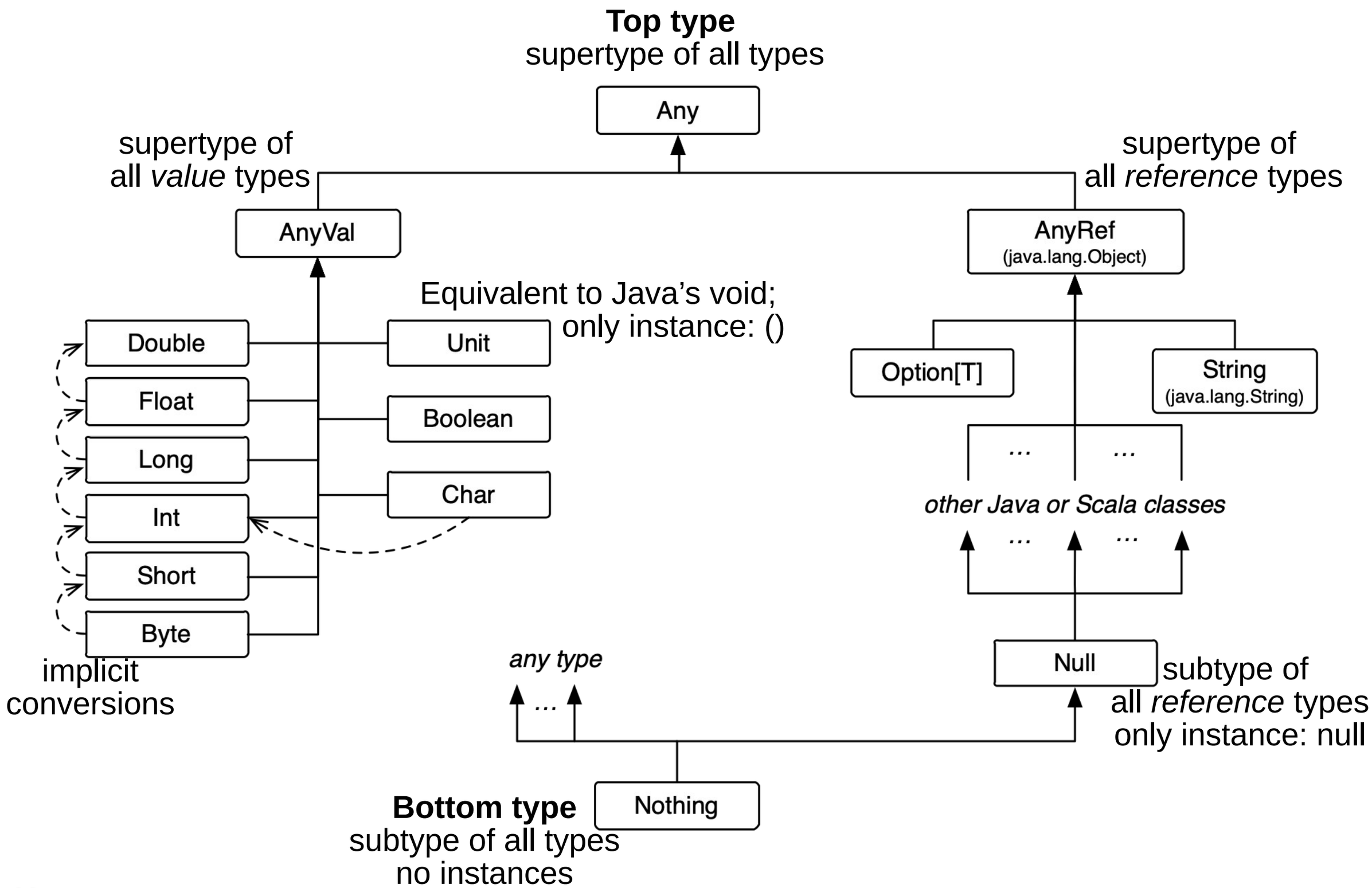
- Distinguish between values defined with different constructors
- Attempt to match a value to a series of patterns
- As soon as a pattern matches, extract and name the parts
- Evaluate code that makes use of these named parts

```
def sum(t: Tree): Int =
  t match {
      case Leaf(n) => n
      case Node(left, right) => sum(left) + sum(right)
}

val t1 = Leaf(3)
val t2 = Node(Node(Leaf(3),Leaf(4)), Leaf(7))

sum(t1)  3: scala.Int
sum(t2)  14: scala.Int
```

Scala's Type Hierarchy



THE TEACHING APPROACH OF COPL

- Basics by the example of arithmetic expressions
- A glimpse of our explorative approach
- Which languages do we study

THE TEACHING APPROACH OF COPL

- Basics by the example of arithmetic expressions
- A glimpse of our explorative approach
- Which languages do we study

Modeling Syntax

- Different notations for the idealized action of adding the idealized numbers (represented) by “3” and “4”:

`3 + 4`	(infix)	Java
`3 4 +`	(postfix)	FORTH
`(+ 3 4)`	(parenthesized prefix)	Scheme

- Ignoring details of concrete syntax, the essence is a tree (AST) …
- First question to answer in modeling languages is how to represent ASTs.

Case Classes for Representing ASTs

Modeling AST for arithmetic expressions with case classes

```
abstract class Expr

case class Num(n: Int) extends Expr
case class Add(lhs: Expr, rhs: Expr) extends Expr
case class Sub(lhs: Expr, rhs: Expr) extends Expr

Add(Num(3),Num(4))
Add(Sub(Num(3),Num(4)),Num(7))
```

Concrete Versus Abstract Syntax

- Concrete syntax
  - What the programmer writes
  - Comments, multiple spaces, newlines, …

- Abstract syntax
  - Internal representation of the syntax (structure of sentences)
  - Smaller than concrete syntax to make automatic processing (e.g., type checking) and reasoning easier.

```
<AE> ::=   <num>
         | Plus  (<AE>, <AE>)
         | Minus (<AE>, <AE>)
```

The AE Interpreter

```scala
def interp(expr: Expr): Int =
  expr match {
      case Num(n) => ???
      case Add(lhs, rhs) => ???
      case Sub(lhs, rhs) => ???
}
```

What goes into “???”

```scala
def interp(expr: Expr): Int =
  expr match {
      case Num(n) => n
      case Add(lhs, rhs) => interp(lhs) + interp(rhs)
      case Sub(lhs, rhs) => interp(lhs) - interp(rhs)
}
```

- Note that we inherit the meaning of + and – as well as the representation of numbers from Scala.
- It is a design decision to do so.

Note…

We write interpreters for specific languages that we define.

But they have the fundamental structure of interpreters for any expression-oriented language that can be used to write programs on a sequential machine.

THE TEACHING APPROACH OF COPL

- Basics by the example of arithmetic expressions
- A glimpse of our explorative approach
- Which languages do we study

An Example Program to Evaluate

```
1   def f (x: Int) = x + 1
2   def g (z: Int) = f(z+4)
3   def h (z:Int, w: Int) = g(z) + g(w)
4
5   f(2)
6   g(5)
7   h(6,7)
```

Let‘s Play Interpreter by Hand …

```
1  def f (x: Int) = x + 1
2  def g (z: Int) = f(z+4)
3  def h (z:Int, w: Int) = g(z) + g(w)
4
5  f(2)
6  g(5)
7  h(6,7)
```

```
→ f(2)
→ x + 1, where x is replaced by 2
→ 2 + 1
→ 3
```

Exploring Alternative Semantics

```
def f (x: Int) = x + 1
def g (z: Int) = f(z+4)
def h (z:Int, w: Int) = g(z) + g(w)

f(2)
g(5)
h(6,7)
```

```
→ g(5)
→ f(z + 4), where z is replaced by 5
→ f(5 + 4)
→ f(9)
→ x + 1, where x is replaced by 9
→ 9 + 1
→ 10
```

Do we have other choices?

Exploring Alternative Semantics

```
→ g(5)
→ f(z + 4), where z is replaced by 5
→ f(5 + 4)
→ f(9)
→ x + 1, where x is replaced by 9
→ 9 + 1
→ 10
```

```
→ g(5)
→ f(z + 4), where z is replaced by 5
→ f(5 + 4)
→ x + 1, where x is replaced by 5 + 4
→ 5 + 4 + 1
→ 10
```

Eager versus lazy evaluation:

- For now, both produce the same answer.
- But this is a very consequential decision!
- In fact one of the most profound choices in PL design.

Exploring Alternative Semantics

```
def f (x: Int) = x + 1
def g (z: Int) = f(z+4)
def h (z:Int, w: Int) = g(z) + g(w)

f(2)
g(5)
h(6,7)
```

```
→ h(6, 7)
→ g(6) + g(7)
→ ...
→ f(6 + 4) + g(7)
→ f(10) + g(7)
→ (10 + 1) + g(7)
→ ...
```

Again choices to make:

- Replace both calls at once, or do one at a time?
- If the latter, do the left or the right one first?

Note…

Because programming languages are artificial and programming is an almost infinitely malleable material, even “standard” features have historically seen variation.

We will either present and discuss alternative semantics in the lecture or encourage you to do so in exercises.

THE TEACHING APPROACH OF COPL

- Basics by the example of arithmetic expressions
- A glimpse of our explorative approach
- Which languages do we study

Study Subject: Core Concepts of PLs

- CoCoPL = Computational core of many widely-used PLs (e.g., eager evaluation, first-class lexically-scoped functions, first-order mutable variables, first-class mutable structures).
- What do concepts mean? How do they work? How can we implement them? How do they interact with each other
- Concepts are building blocks of languages. How many of them do we need? Can we build a language from a minimal “core”? What are the tradeoffs of doing so?

Focus on Core Concepts

Photo by Mukul Wadhwa on Unsplash

Forget all the concrete features of languages you know …

We look at core concepts … and we define the meaning of even most trivial ones

Note …

- What belong to CoCoPL is somewhat a subjective decision.
- Will consider also some non-“standard” features that are not only important, but also beautiful and fascinating (according to our subjective view), e.g., lazy and reactive evaluation, differentiable functions, etc.

Summarizing Remarks About COPL

- What this course **IS**
  - About **concepts in PL**
  - About **interpreters**
  - About deeply **understanding PL abstractions**
  - About bridging with more formal approaches

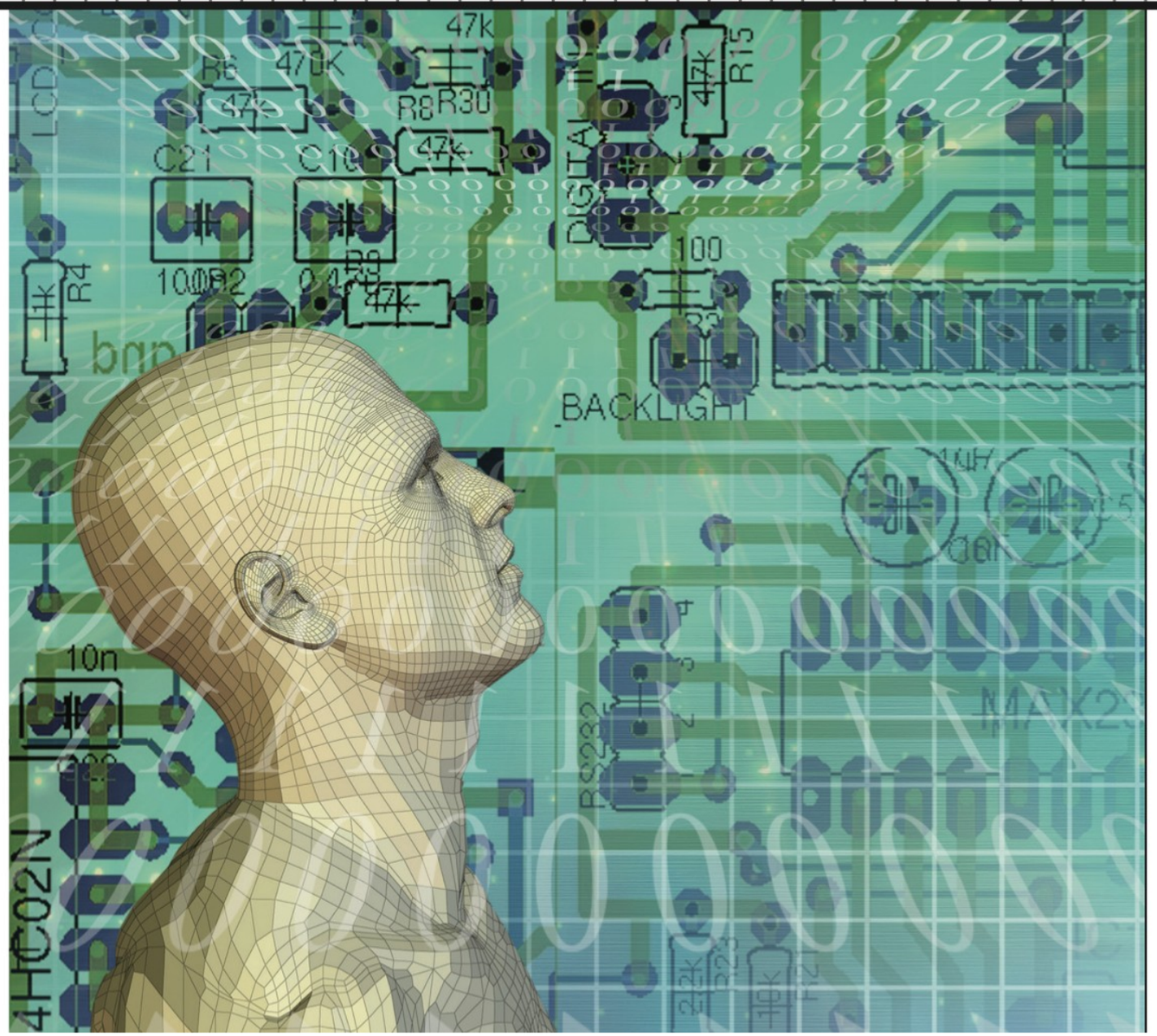
- What this course **IS NOT**
  - A course on Scala
  - A course **on a specific PL**
  - A course **on PL theory** (formal semantics, type theory/systems, …)
  - A course **on PL implementation**: Code optomizations, Intermediate representations, virtual machines, …

WHEN YOU “DESIGN” A PL IN 10 DAYS …

COMPUTING CONVERSATIONS

JavaScript: Designing a Language in 10 Days

Charles Severance
University of Michigan



ECMAScript

- A standardized specification of a scripting language developed by Brendan Eich of Netscape
- Initially it was named Mocha, later LiveScript, and finally JavaScript.
- In December 1995, Sun Microsystems and Netscape announced JavaScript in a press release.
- In March 1996, Netscape Navigator 2.0 was released, featuring support for JavaScript.

The Next 20 Years …

- Sergio Maffeis, John C. Mitchell, and Ankur Taly. 2008. **An Operational Semantics for JavaScript.** In *Proceedings of the 6th Asian Symposium on Programming Languages and Systems* (APLAS '08), G. Ramalingam (Ed.). Springer-Verlag, Berlin, Heidelberg, 307-325. DOI=http://dx.doi.org/10.1007/978-3-540-89330-1_22

- Arjun Guha, Claudiu Saftoiu, and Shriram Krishnamurthi. 2010. **The essence of javascript**. In *Proceedings of the 24th European conference on Object-oriented programming* (ECOOP'10), Theo D'Hondt (Ed.). Springer-Verlag, Berlin, Heidelberg, 126-150.

- Daejun Park, Andrei Stefănescu, and Grigore Roşu. 2015. **KJS: a complete formal semantics of JavaScript**. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (PLDI '15). ACM, New York, NY, USA, 346-356. DOI: http://dx.doi.org/10.1145/2737924.2737991

- …

The Result …

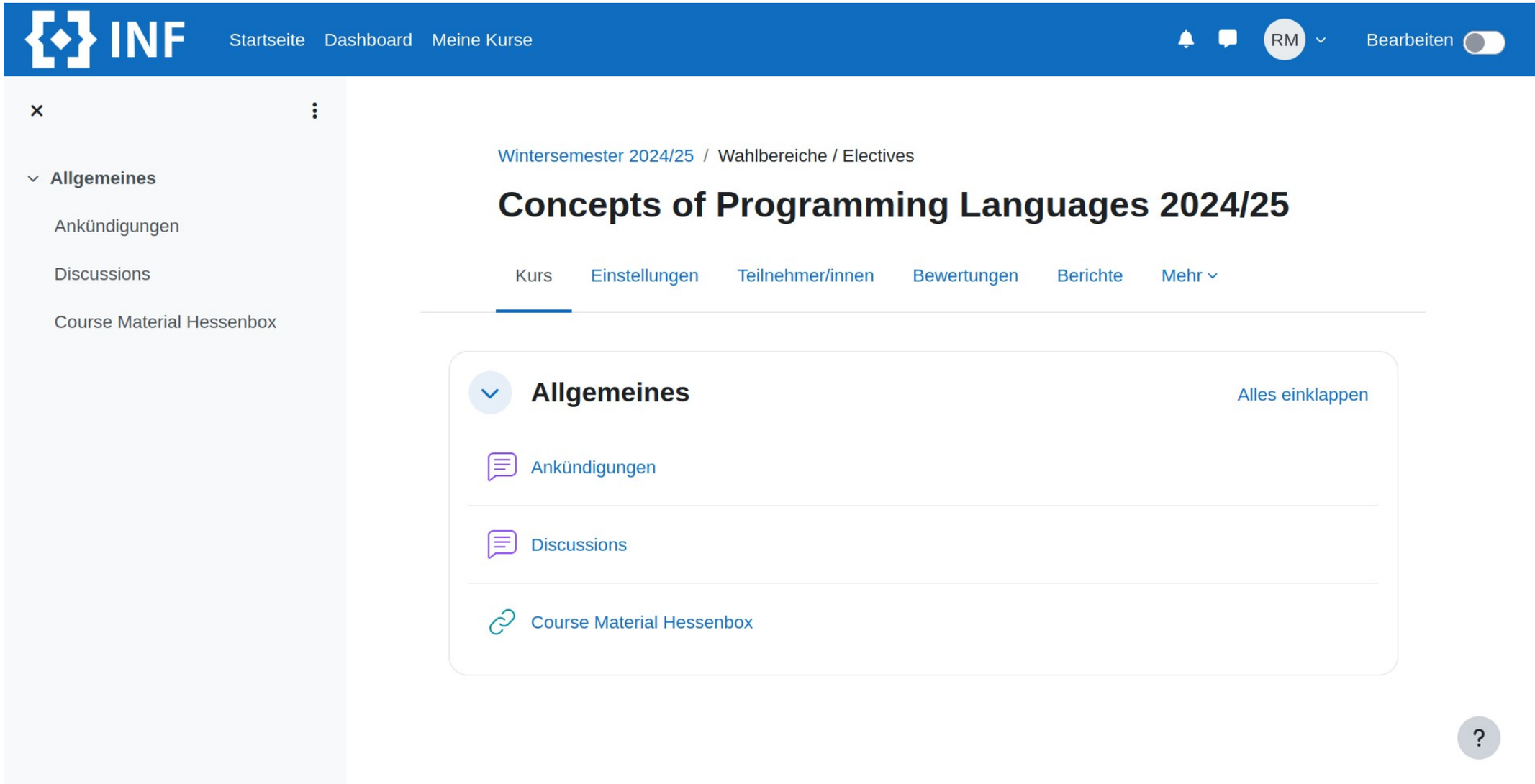
Let's check a talk by Gary Bernhardt from CodeMash 2012

https://www.destroyallsoftware.com/talks/wat

ADMINISTRATIVA

Course Moodle

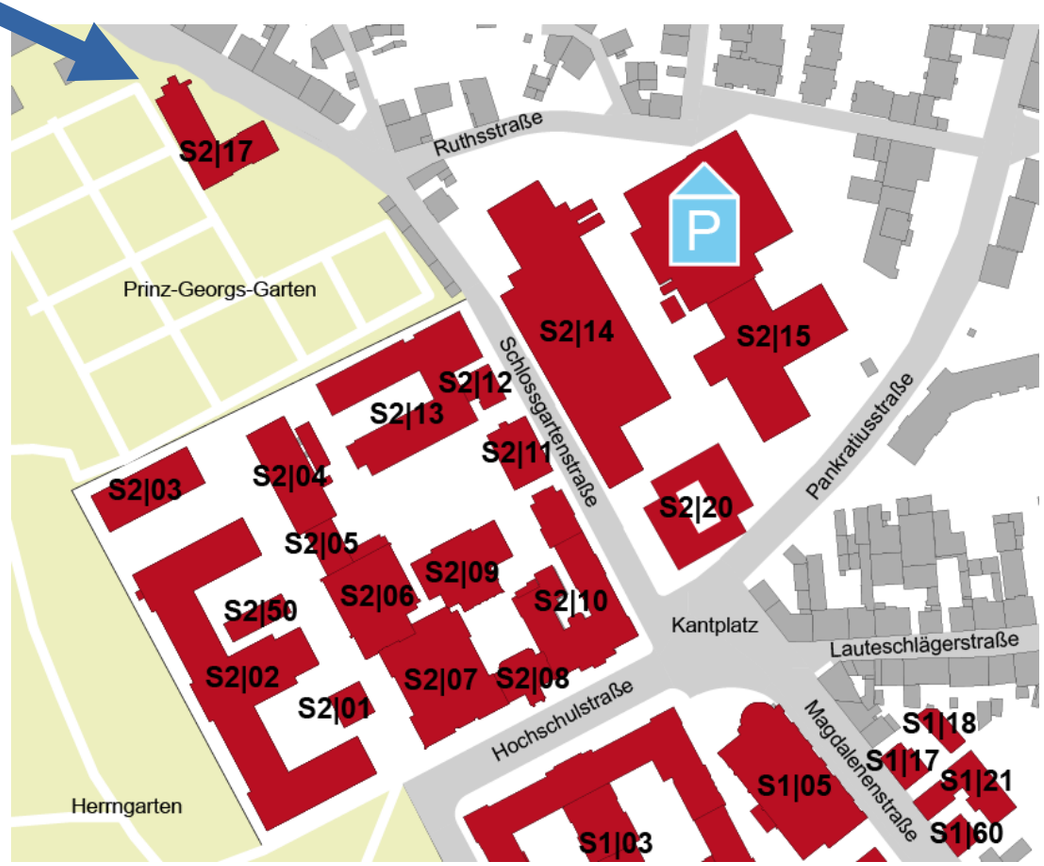
- Announcements
- Course material & exercises
- Ask questions



Exercises and Q&A Session

- Thursday 11:40–13:20 in S217/103
- Understanding exercises is central to the lecture
- Homework exercises

- The schedule lecture/exercise can change depending on the week



Written Exam

Date: Wed, 11. Mar. 2026 09:00-12:00

What?

- Implement/extend interpreters
- “Theory”
- Understanding programs under different semantics
- More about this towards the end of the course…

Course Material Beyond Slides

plai.org

Programming Languages: Application and Interpretation | plai

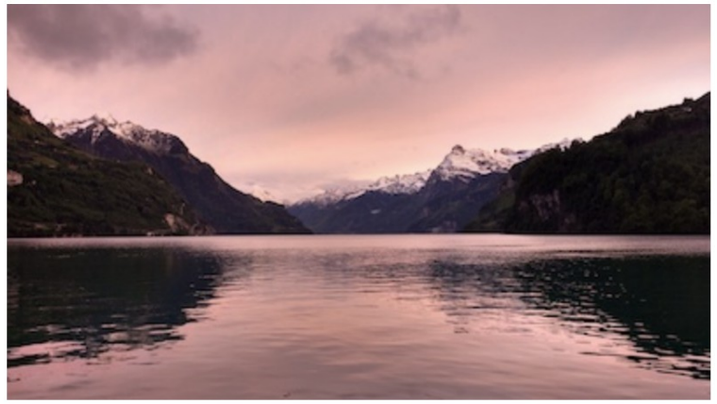
plai

Programming Languages: Application and Interpretation

This is the Web site for *Programming Languages: Application and Interpretation*, often referred to by its initials as PLAI (pronounce it like “play”). Over the years well over fifty academic institutions (that I know of) have used PLAI.

PLAI is designed for upper-level courses that introduce the main ideas of programming languages. In the US, it is designed for students in their second- through fourth-years of college, as well as starting graduate students. However, PLAI has been used with students much younger, including in a few select high school classes.

The Current (Third) Edition



The third edition finally came out in December 2022! For the foreseeable future I will be putting out two versions, “electronic” and “print”. The former has hyperlinks in its Table of Contents, while the latter has page numbers. Unfortunately, cross-references within the latter document are still hyperlinks (blame Google Docs), and will hence likely be useless on paper. Sorry.

Other Courses on PL

- Design and Implementation of Modern Programming Languages (Seminar & Project)
  - Scientific reading and writing
  - Practical implementations

- Software Engineering Project (see **next slide**)

- See our website https://www.stg.tu-darmstadt.de/ for details.

Other Courses by Our Lab

QUESTIONS?